



University
of Glasgow

Lecture 2: Secured SDLC Security Requirement Planning

A WORLD
TOP 100
UNIVERSITY

Dr. Yutian Tang

Yutian.Tang@glasgow.ac.uk

WORLD
CHANGING
GLASGOW

THE SUNDAY TIMES
GOOD
UNIVERSITY
GUIDE
2024
SCOTTISH
UNIVERSITY
OF THE YEAR



University
of Glasgow

Attendance

Scan the QR code to take your attendance

Note that this QR code is only valid
today (3:00 PM-6:00 PM)

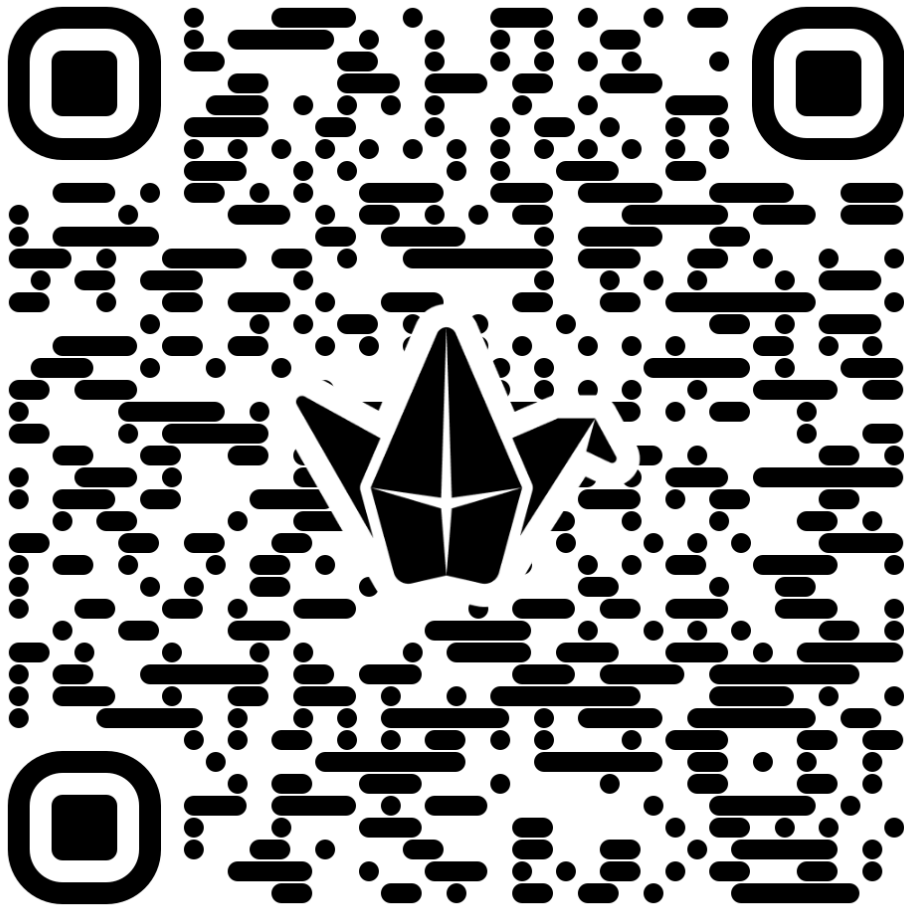




University
of Glasgow

:Padlet for Questions

- <https://padlet.com/yutiantang/sse-lecture-2-question-wall-p3h44ejq6y4t3iyg>



Please post any questions you have during the lecture on this Padlet. I will address them at the end.



University
of Glasgow

Outline

- Vulnerability, Threats, and Countermeasures
- Secured Software Engineering Lifecycles (SSDLC)
- Security Requirement Planning



University
of Glasgow

Vulnerabilities, Threats, Countermeasures



University
of Glasgow

Vulnerabilities



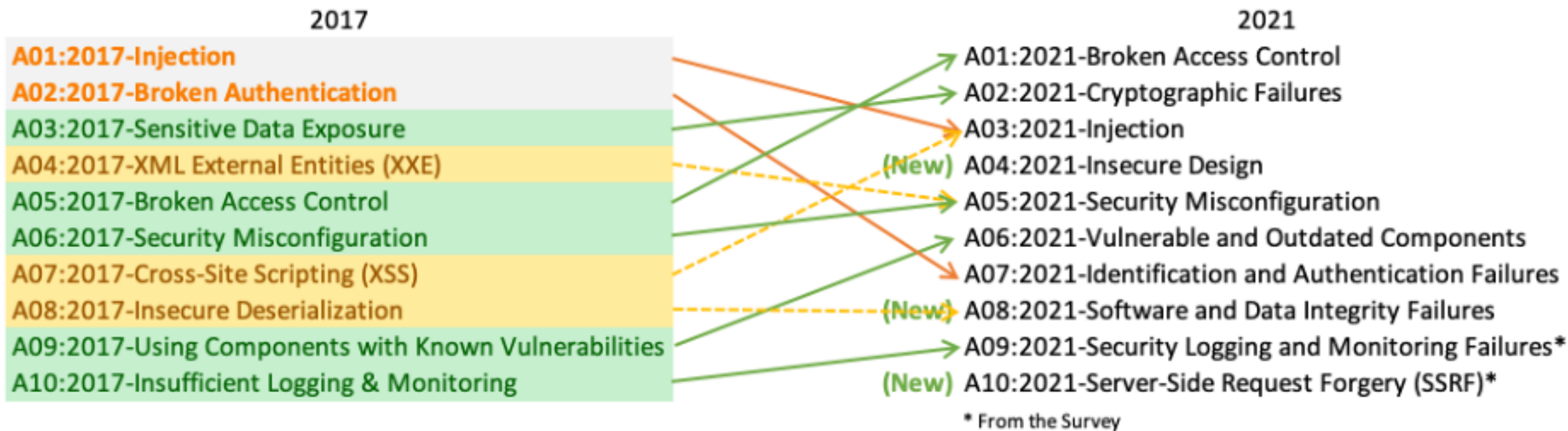
Understanding Vulnerabilities

Definition of Software Vulnerability

- A security flaw, glitch, or weakness found in software code that could be exploited by an attacker.

Understanding Vulnerabilities'3

- Top-ranked Vulnerabilities: OWASP 2021



Top-10 Vulnerability: Broken Access Control



- Access control enforces policy such that users **cannot act outside of their intended permissions.**
- Failures typically lead to unauthorized information disclosure, modification, or destruction of all data or performing a business function outside the user's limits.



Top-10 Vulnerability: Broken Access Control



Name: Alice

UID: 12345

<https://www.bank.com/account?number=12345>



(Alice is authenticated user for her account)

<https://www.bank.com/account?number=23456>



(Alice should not be authenticated for this page)



Top-10 Vulnerability: Broken Access Control



Name: Alice

UID: 12345

<https://www.bank.com/account?number=12345>



(Alice is an authenticated user for her account)

<https://www.bank.com/account?number=23456>



(Alice should not be authenticated for this page)

Broken Access Control: Alice can access and manipulate accounts of other users (e.g., id=23456).

Common access control vulnerabilities include:

1. Violation of the principle of **least privilege or deny by default**, where access should only be granted for particular capabilities, roles, or users, but is available to anyone.

- <https://www.bank.com/account?number=12345> (✓)
- Alice with ID=12345
- <https://www.bank.com/admin> (X)

Common access control vulnerabilities include:

2. Bypassing access control checks by modifying the URL, internal application state, or the HTML page, or by using an attack tool to modify API requests.

- A user logs in to his/her account (id = 123) and clicks on a link to view his/her document, which takes him/her to the URL: <https://example.com/files/123.pdf>.
- If we change the URL to <https://example.com/files/>, can we get all user files?

Common access control vulnerabilities include:

3. Accessing API with missing access controls for POST, PUT and DELETE.

- To update an existing article, a user sends a PUT request to https://blogplatform.example.com/api/posts/{post_id} with revised content.
- The platform must verify the user is authorized to send the PUT request. (he/she should be the owner of the post).



Case Study

A user Alice (uid: 12345) is authenticated and can successfully access her bank account page at:

<https://www.bank.com/account?number=12345>

She then manually changes the URL to:

<https://www.bank.com/account?number=23456>

The system responds with a page stating:

“There is no such user.”

Does this break access control?



Join at menti.com | use code 2135 5612



Menti

Lecture 2: SSDLC



... The system responds with a page stating: "There is no such user." Does this break access control?



Choose a slide to present

Slide 1: ... The system responds with a page stating: "There is no such user." Is there a security problem in the system?

0 Yes 0 No

Slide 2: After deployment, a system does not check whether its code or configuration files have been changed. What is the main problem with this situation?

0 cannot ensure that its code and data have not been tampered with 0 cannot prevent users from logging in 0 may become unavailable 0 cannot send data securely over the network

Slide 3: What best explains why a DoS attack is possible in this scenario?

0 A the system was designed without considering its capacity limits 0 B the attacker successfully bypassed user authentication 0 C the system failed to encrypt network communications 0 D the attacker disabled the system's source code

Slide 4: A development team has limited time and budget. What is the most realistic goal of secure software engineering in this situation?





Top-10 Vulnerability: Injection



- Injection refers to a security flaw where attackers insert malicious code or commands into software applications, causing them to execute unintended operations.
- The malicious data can trick the interpreter to executing unintended commands or accessing data without authorization.
- Injection vulnerabilities can occur in various contexts, such as SQL, NoSQL, OS command, and XPath queries.



Top-10 Vulnerability: Injection'2



Case Study:

- Username, password;
- SQL query;
- A simplified version:
 - `SELECT * FROM users WHERE username = '[user_input]' AND password = '[password_input]';`
- The user inputs his/her username and password, which are safely enclosed in the query.

Case Study:

- **Injection Attack Example:**

- An attacker wants to log in without knowing valid credentials.
- He/she enters the following in the username field: ' OR '1'='1' -- AND password = 'whatever'

- The resulting SQL query:

```
SELECT * FROM users WHERE username = " OR '1'='1' --' AND password = 'whatever';
```



-- 'starts a comment, which means that the rest of the line will be ignored by the SQL server

```
SELECT * FROM users WHERE username = " OR '1'='1'
```

Top-10 Vulnerability: Insecure Design



- Insecure design refers to a spectrum of vulnerabilities due to **lacking or weak protective measures** within the system's architecture.
- Example:
 - Poor Authentication Mechanisms
 - Simple password-based authentication without multi-factor authentication.
 - Passwords stored in plaintext in the database.
 - Insecure Communication
 - Sensitive data transmitted over HTTP, not HTTPS.



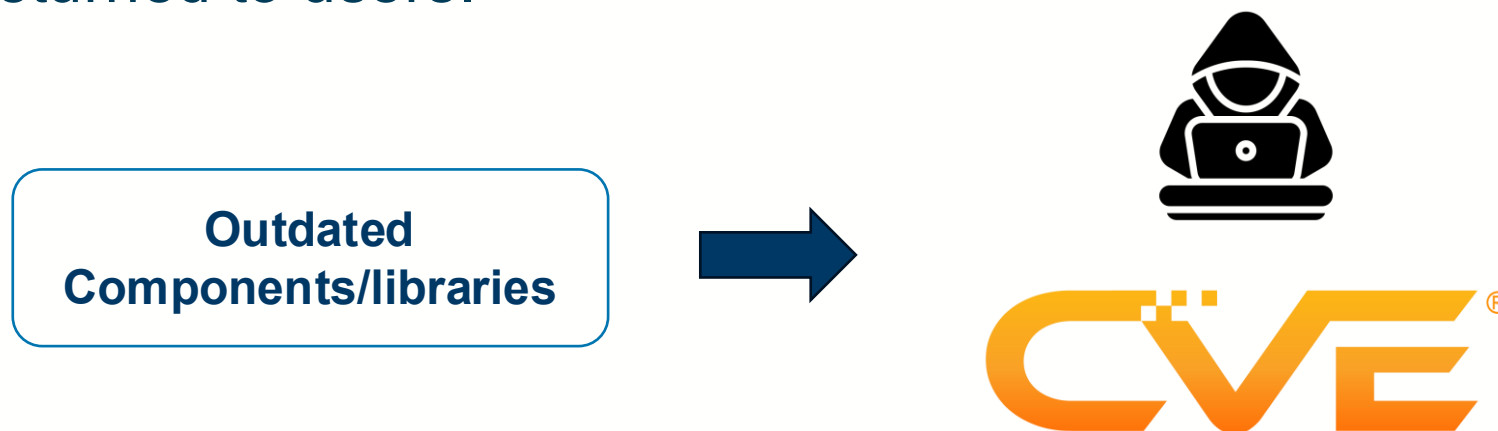
Top-10 Vulnerability: Security Misconfiguration



- Security Misconfiguration is defined as any error or vulnerability present in the configuration of code that allows attackers access to sensitive data.
- This can include a wide range of issues, from improper security settings to maintaining default configurations, and can affect any layer of the application stack, cloud, or network

Case Study

- The application server's configuration allows detailed error messages, e.g., stack traces, to be returned to users.



- This potentially exposes sensitive information or underlying flaws such as component versions that are known to be vulnerable.

Top-10 Vulnerability: Vulnerability and Outdated Component



You are likely vulnerable:

- If you do not know the versions of all components you use (both client-side and server-side).
- If the software is vulnerable, unsupported, or out of date.
- If you do not scan for vulnerabilities regularly and subscribe to security bulletins related to the components you use.
- If software developers do not test the compatibility of updated, upgraded, or patched libraries.



Top-10 Vulnerability: Identification and Authorization Failure



- Confirmation of the user's identity, authentication, and session management is critical to protect against authentication-related attacks.
- There may be authentication weaknesses if the application:
 - Permits default, weak, or well-known passwords, such as "Password1" or "admin".
 - Uses weak or ineffective credential recovery and forgot-password processes, such as "knowledge-based answers," which cannot be made safe.
 - When is your birthday?
 - What was your first car?
 - Has missing or ineffective multi-factor authentication.
 - Exposes session identifier in the URL, reuse session identifier after successful login.
 - ...



Top-10 Vulnerability: Identification and Authorization Failure



- **Example**

- Application **session timeouts aren't set correctly.**
- A user uses a public computer to access an application.
- Instead of selecting "logout," the user simply closes the browser tab and walks away.
- **An attacker uses the same browser an hour later, and the user is still authenticated.**



Top-10 Vulnerability: Software and Data Integrity Failures



- Software and data integrity failures relate to **code and infrastructure that does not protect against integrity violations**.
 - E.g. Rely on plugins, libraries, or modules from **untrusted** sources
- Integrity is about whether the system can trust that its code and data remain unchanged.
- Attackers can exploit these vulnerabilities to gain access to sensitive information or cause damage to the system.

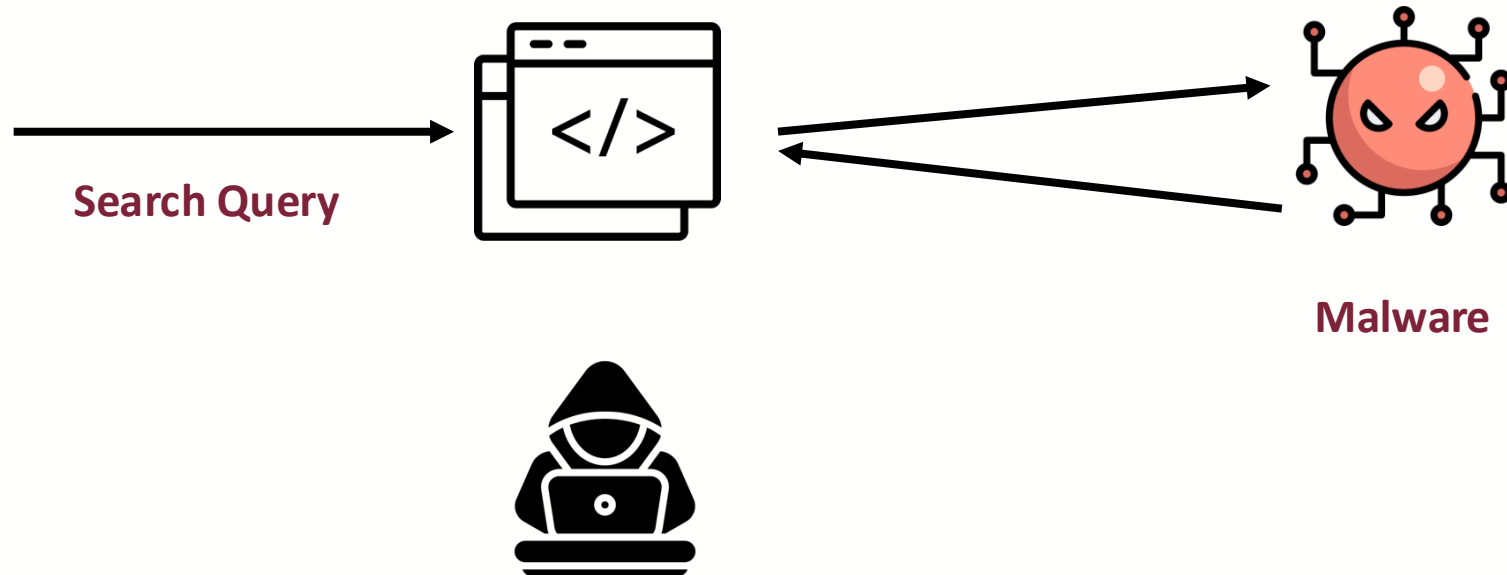


Top-10 Vulnerability: Software and Data Integrity Failures



- **Example (CWE: 426)**

- The product searches for critical resources using an externally-supplied search path that can point to resources that **are not under the product's direct control**.





Case Study

After deployment, a system does not check whether its code or configuration files have been changed.

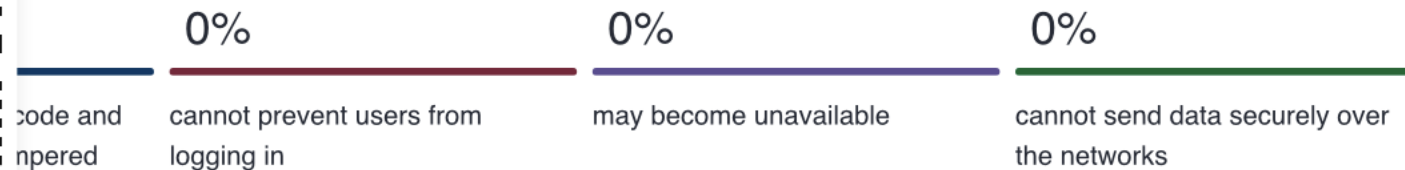
What is the main problem with this situation

- A. The system cannot ensure that its code and data have not been tampered with
- B. The system cannot prevent users from logging in
- C. The system may become unavailable
- D. The system cannot send data securely over the networks

Join at menti.com | use code 2135 5612



After deployment, a system does not check whether its code or configuration files have been changed. What is the main problem with this situation?

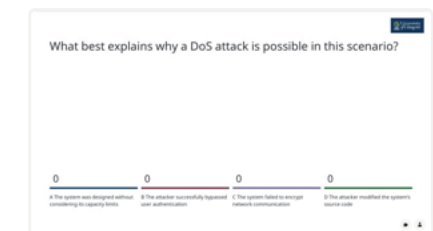
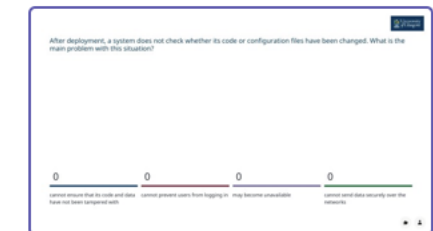
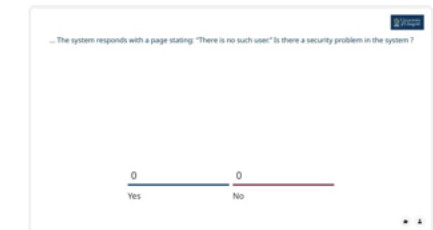


Menti

Lecture 2: SSDLC



Choose a slide to present





Threats and Vulnerabilities

- **Nature:** **Vulnerabilities** are inherent flaws or weaknesses;
 - **Threats** are potential or actual actions that can exploit the vulnerabilities.
- **Dependency:**
 - A threat does not need a vulnerability to exist (e.g., natural disasters); however, most software/cyber threats exploit existing vulnerabilities.



Threat Agents'2: Non-target specific Threats Agents

- Non-target specific threats agents:

- Virus
- Worm
- Trojan
- Backdoor

Threat Agent: This is the means or tool used by the threat actor to execute the attack.

- Logic Bombs
- Phishing
- Denial-of-Service (DoS) Attacks
- ...



Backdoor

- **Definition:** A backdoor is an illicitly created or pre-existing access point in a system that enables an unauthorized party to access the system without going through the normal authentication and authorization processes. (NIST (National Institute of Standards and Technology))
- This can be used by attackers to
 - steal data,
 - install malware, or
 - carry out other malicious activities.



How Backdoors Work

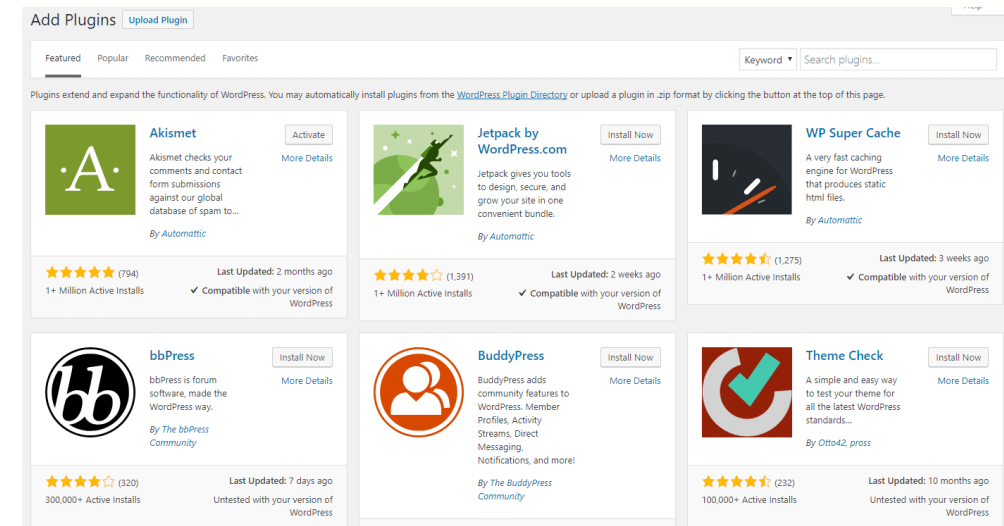
- **Creation and Installation:** Backdoors can be part of legitimate software, installed by hackers, or introduced via malware.
- **Functioning:** Serve as hidden mechanisms for unauthorized system access, often used for stealing sensitive information or installing other types of malware.



Backdoor Example

- **Example**

- One of the greatest threats that Wordpress faced is the injection of malicious code in its JavaScript or PHP code.
- Security researchers found that **some adversaries plant a backdoor to make changes to the site's code and gain future access.**



Img:

<https://kinsta.com/knowledgebase/wordpress-plugin/>

7 most common application backdoors

<https://resources.infosecinstitute.com/topics/application-security/7-most-common-application-backdoors/>



Phishing

- Phishing is a form of social engineering attack.
- Attackers use deceptive emails and forged websites to carry out online fraud activities.



Email Phishing

- Email phishing is a malicious cyber-attack method within social engineering.
- Email phishing is when a cyber attacker sends you an email pretending to be someone else in hopes that you'll reply with the information they requested.



Email Phishing Example

- **Example 1:** One phishing email example is a cybercriminal emailing you while pretending to be your relative.

Fake emails



Simon Gay <Simon.Gay@glasgow.ac.uk>



Tue 2024-01-16 16:11

Dear All,

There have been a number of fake emails this afternoon, to various members of the School, using my name but with a different email address. If you receive any such email, please ignore it, and certainly don't reply because it will probably lead to a phishing attack.

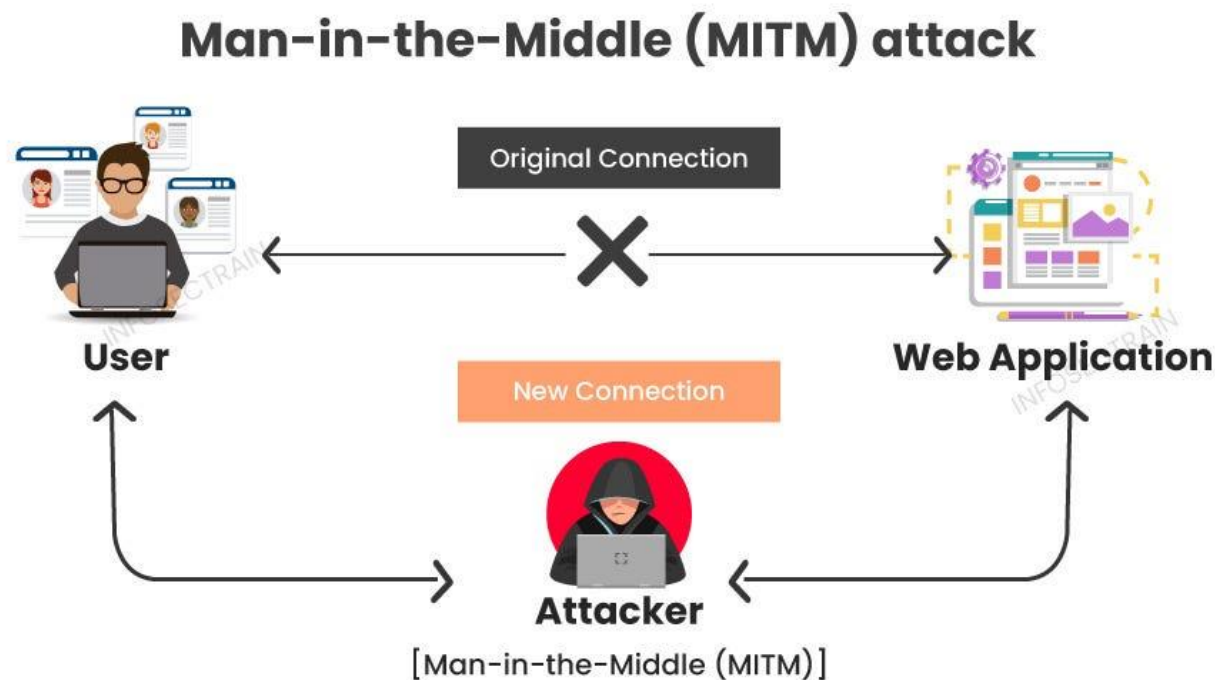
Simon

← Reply

→ Forward

Man-in-the-middle (MITM) attacks

- **Definition:** A man-in-the-middle (MITM) attack is an attack in which a hacker steals your information by getting in between you and a trusted party.
- If successful, the hacker may be able to gain access to your internet traffic and personal information.





Denial-of-Service (DoS) Attacks

- **Def:**
- A **Denial-of-Service (DoS)** attack is a malicious attempt to disrupt the normal functioning of a targeted server, service, or network by overwhelming it with a flood of traffic or requests.
- This results in the victim's resources being exhausted, rendering the service inaccessible to legitimate users.



DoS Attack Example

- **Example:**
 - Small online bookstore
 - 100 users simultaneously
 - Attacker-→ thousands of requests per second to the bookstore's server

What best explains why a DoS attack is possible in this scenario?



- What best explains why a DoS attack is possible in this scenario?
- A The system was designed without considering its capacity limits
- B The attacker successfully bypassed user authentication
- C The system failed to encrypt network communication
- D The attacker modified the system's source code

Join at menti.com | use code 2135 5612



What best explains why a DoS attack is possible in this scenario?



ed without
its

0

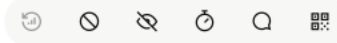
B The attacker successfully bypassed user authentication

0

C The system failed to encrypt network communication

0

D The attacker modified the system's source code

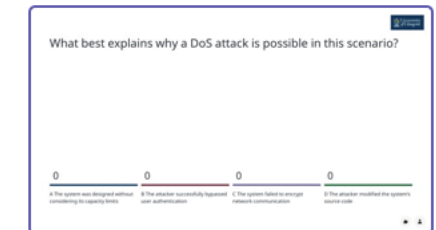
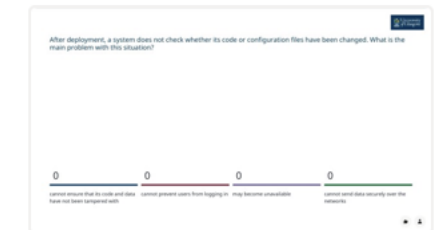
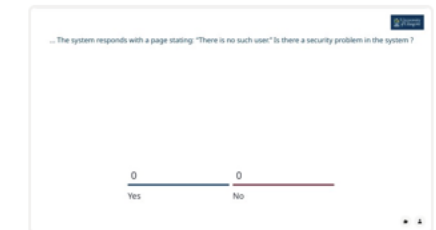


Menti

Lecture 2: SSDLC



Choose a slide to present





DoS Attack Example

- **Example:**
 - Small online bookstore
 - 100 users simultaneously
 - Attacker-→ thousands of requests per second to the bookstore's server

DoS

Threats



What best explains why a DoS attack is possible in this scenario?

**Capacity
limits**

**Vulnerabilities
(root cause)**



Countermeasures

- **Importance of Countermeasures**

- Essential for risk management, protecting assets, compliance, and maintaining user trust.

- **Types of Countermeasures:**

- Preventive (before attacks),
- Detective (identifying threats), and
- Responsive (post-incident actions)



Case Study

A development team has limited time and budget. What is the most realistic goal of secure software engineering in this situation?

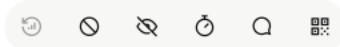
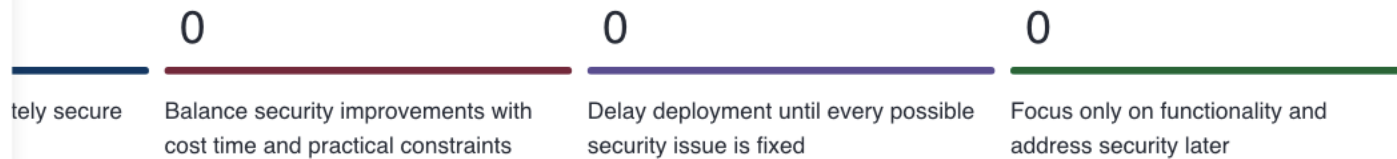
- A. Make the software completely secure by eliminating all risks
- B. Balance security improvements with cost time and practical constraints
- C. Delay deployment until every possible security issue is fixed
- D. Focus only on functionality and address security later



Join at menti.com | use code **2135 5612**



A development team has limited time and budget. What is the most realistic goal of secure software engineering in this situation?

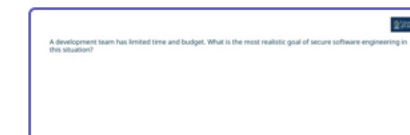
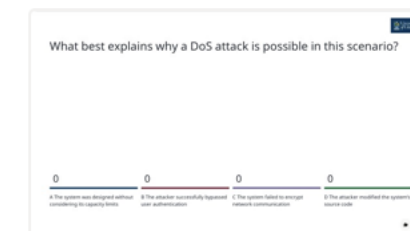
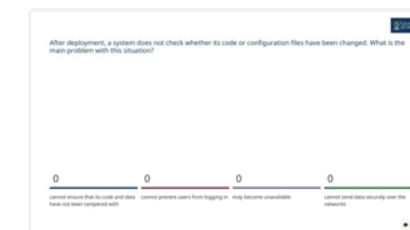
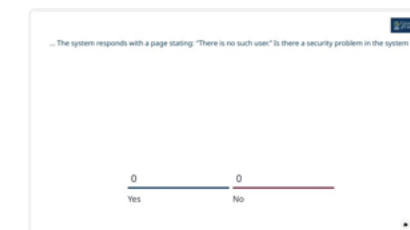


Menti

Lecture 2: SSDLC



Choose a slide to present





Bad News !

- As a general rule, there is no way to make a software completely secure!
- Tradeoff between allocating resources to make the software secure vs compromise
 - Cost of implementing the countermeasures?
 - Risks?



University
of Glasgow

End of the Story?



Best Practices in Countermeasure Implementation

- Integrating security in Software Development Lifecycle (SDLC)
- Training and Awareness
- Continuous Evaluation and Updating



University
of Glasgow

Lecture 2: Secured SDLC Security Requirement Planning

A WORLD
TOP 100
UNIVERSITY

Dr. Yutian Tang

Yutian.Tang@glasgow.ac.uk

We are resuming at :

WORLD
CHANGING
GLASGOW

THE SUNDAY TIMES
GOOD
UNIVERSITY
GUIDE
2024
SCOTTISH
UNIVERSITY
OF THE YEAR



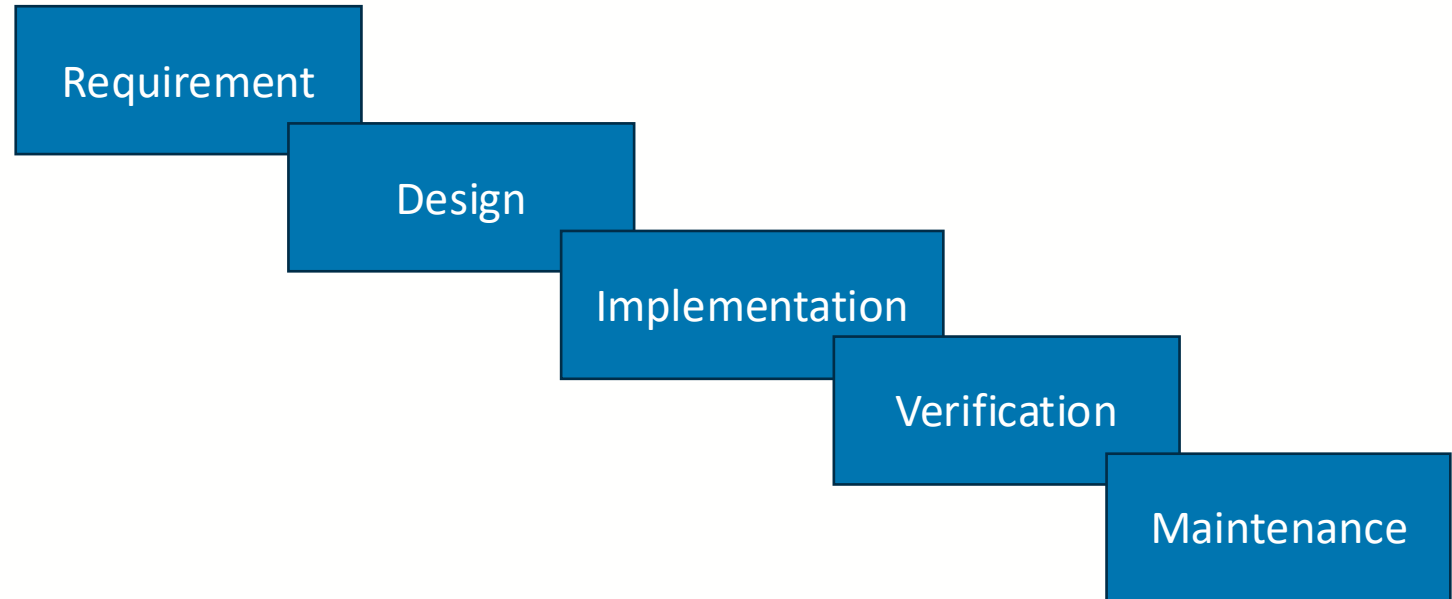
University
of Glasgow

Secured Software Development Lifecycle



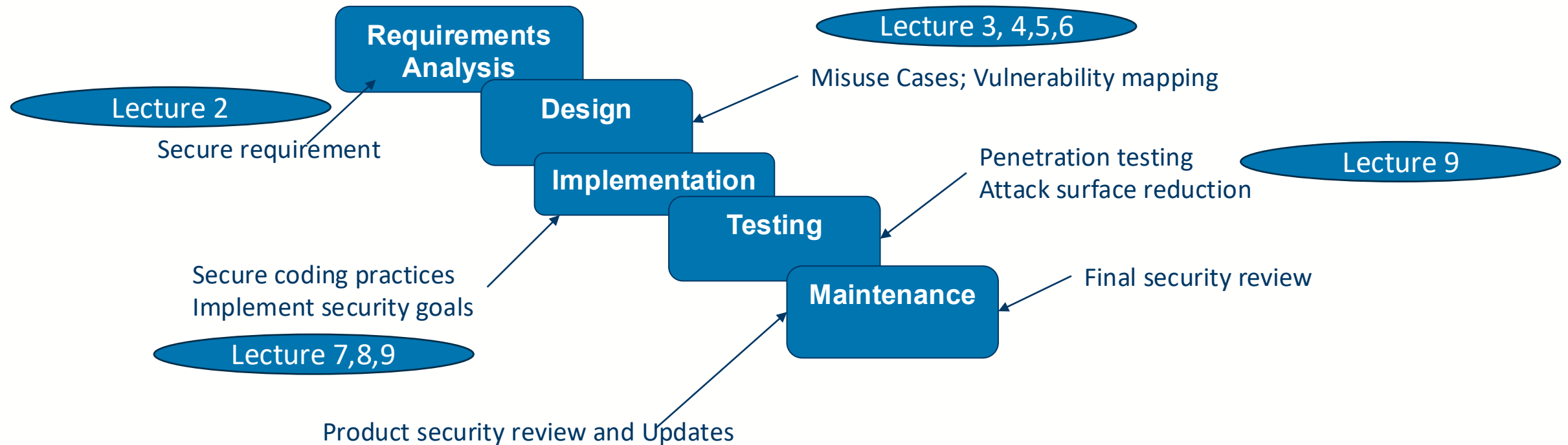
Software Development Lifecycles (SDLC)

- Overview of SDLC phases
- Integrating security in each phase:
 - Requirements
 - Design
 - Implementation
 - Verification
 - Maintenance



Secured Software Development Lifecycles (SDLC)

- Overview of Secure SDLC phases



Waterfall model with security considerations



Security in the Planning & Requirement Phase

Requirement

- **Importance of security in the early stages**
- Identifying requirements with security concerns



Secure Design Principles

- **Security by Design concept**
 - A proactive approach that integrates security measures from the onset of the software development process rather than as an afterthought.
- Key principles (e.g., Least Privilege, Defense in Depth)
 - Key Design Principles (see Lecture 1)



Implementing Security in Development

Implementation

- Secure coding practices
 - Secure coding patterns
 - Secure design
- Security testing methodologies (e.g., Static and Dynamic Analysis)
 - **Static Analysis:** Examination of the application's source code without executing it.
 - **Use Cases:** Early detection of vulnerabilities, code quality checks.
 - **Tools:** SonarQube, Fortify, Checkmarx.



Implementing Security in Development

Implementation

- Secure coding practices
 - Secure coding patterns
 - Secure design
- Security testing methodologies (e.g., Static and Dynamic Analysis)
 - **Dynamic Analysis:** Testing the application while it is running, from the outside.
 - **Use Cases:** Identifying vulnerabilities in a running application, penetration testing.
 - **Tools:** OWASP ZAP, Burp Suite, AppScan.



- Security in the testing phase
 - **Penetration Testing:** Assesses the security and weaknesses of vehicle information technology systems by simulating real attack behaviors. (ISO 21434)
 - **Security Regression Testing:** Ensure that changes do not introduce new vulnerabilities.
 - **Compliance Testing:** The process of verifying that the software adheres to the relevant security standards and regulations
 -



University
of Glasgow

Secured Requirement Planning



Gathering Requirements

- Security is important.
- **Impossible** to gather every necessary requirement for a perfect system.
- A **requirement** is the outcome of the proposed system, something it must perform or a quality it must have, **NOT** a specification of how it should accomplish this.
- Investigation on what the proposed system should do and what it should contain.



Types of Requirements

- **Functional requirements**

- Statements of
 - services the system should provide,
 - how the system should react to particular inputs and
 - how the system should behave in particular situations.
- May state what the system should not do.

- **Non-functional requirements**

- Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
- Often apply to the system as a whole rather than individual features or services.



Types of requirements

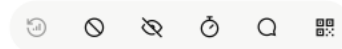
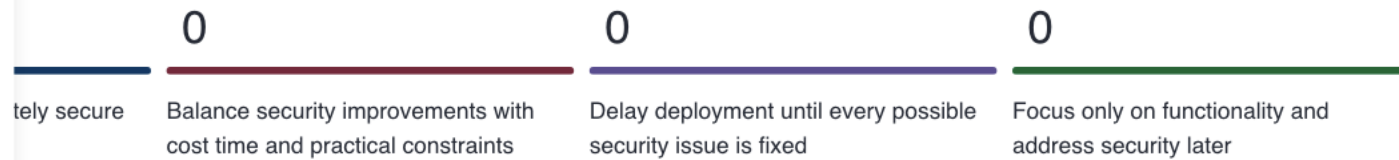
- Functional: functionality of the system
- Non-functional: quality or constraint
- **Security requirement** is an associated protection that must be placed on some part of the system as contingency to normal operation or a guarantee of some constraint what would otherwise violate the conditions of safe operation.
- Security requirement can be **functional or non-functional**
- Encounter security requirements for every requirement is the key



Join at menti.com | use code **2135 5612**



A development team has limited time and budget. What is the most realistic goal of secure software engineering in this situation?

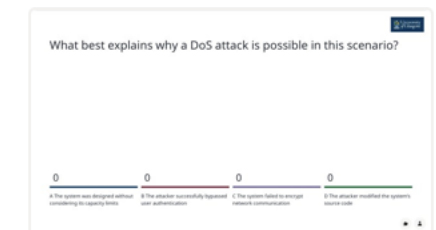
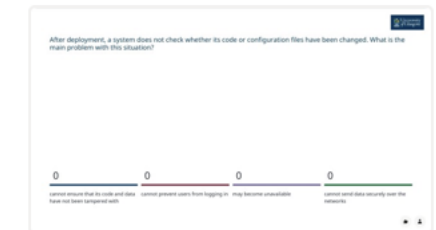
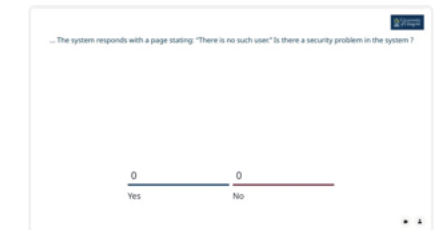


Menti

Lecture 2: SSDLC



Choose a slide to present





Requirements Gathering

- **Methodology:** Requirements will be collected through a blend of structured interviews and comprehensive feedback on the project synopsis provided by key stakeholders.
- **Stakeholder Engagement**
 - Stakeholders will actively participate in a collaborative application design session to ensure all perspectives are integrated.



Requirements Gathering'2

- **System Enhancements: Feedback and Evaluation**
 - **For Upgrades:** Engage in a thorough review of the existing system's documentation, coupled with robust interaction with end users, to enhance the response quality and gather pertinent feedback.
 - **For New Systems:** Develop a working prototype to facilitate hands-on experience and garner user feedback for iterative improvement.
- **Prioritizing Requirements**
 - **Criteria for Ranking:** Requirements will be prioritized according to their relevance to the project goals and the feasibility of implementation, reflecting the consensus of stakeholder input.



Requirements Analysis Process Overview

- **Documentation of Requirements**

- **Recording Medium:** Requirements are initially recorded on cards or paper to capture detailed information and any necessary clarifications directly from stakeholders.



Requirements Analysis Process Overview'2

- **Data Organization**
- **Spreadsheet Entry:** The subsequent phase involves transferring these details to a spreadsheet. This document will include *columns* for:
 - Key Stakeholder
 - Requirement Type (Functional [F] or Non-Functional [NF])
 - Unique Identifier
 - Priority Level
 - Stakeholder Relationship



Requirements Analysis Process Overview'4

- **Prioritization and Feasibility**

- **Requirement Ranking:** Requirements will be evaluated and marked for potential rejection [R].
- **Feasibility Assessment:** Establish a cutoff for requirements based on projected manpower availability, project scope, and associated costs.

- **Security Integration**

- **Security Assessment:** Each requirement must include an analysis of the **security implications for the relevant system component** and ensure that appropriate security measures are integrated.



Requirement Analysis and Risk Assessment

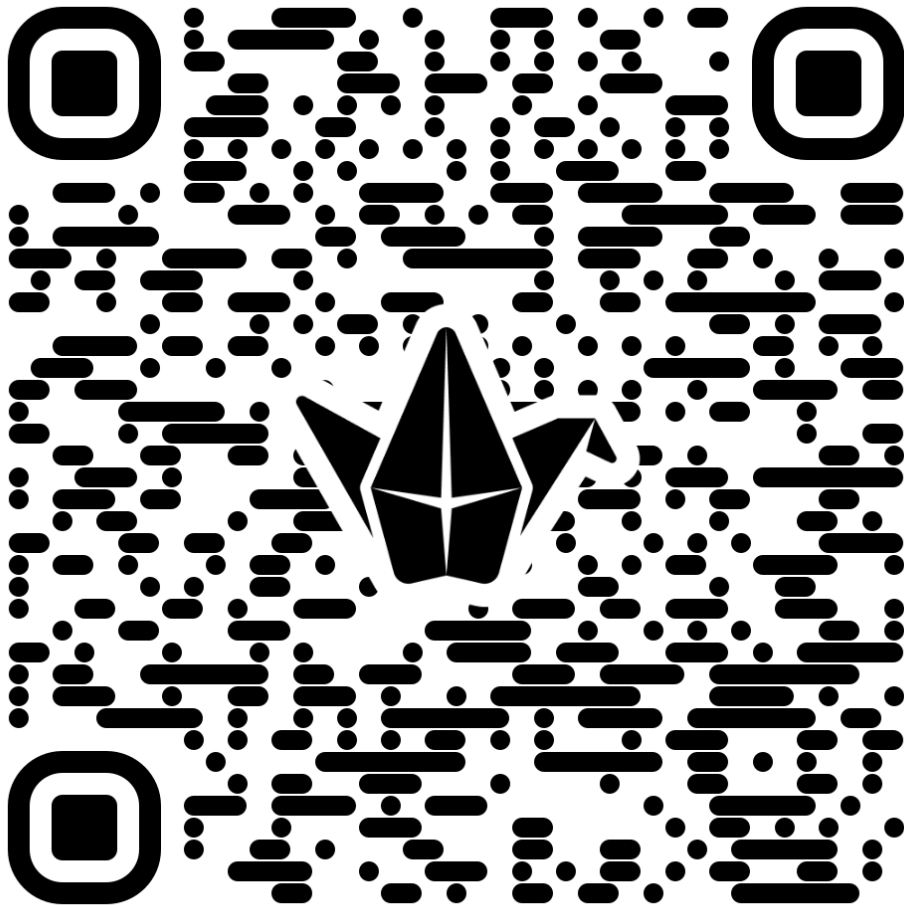
- Why should be this **part of** the system?
- What are the **constraints** on this requirement?
- What are the **dependencies** for this requirement?
- Who are the **stakeholders** for this requirement?
- **Security component**: Fail case; Consequence of failure; associated risks
- What are the **exceptions** to the normal case for this requirement?
- What **sensitive information** is included in this requirement?
- What are the **consequences** if the conditions of this requirement are **violated**?
- What happens if this requirement is **intentionally violated**?



University
of Glasgow

:Padlet for Questions

- <https://padlet.com/yutiantang/sse-lecture-2-question-wall-p3h44ejq6y4t3iyg>



Please post any questions you have during the lecture on this Padlet. I will address them at the end.



Acknowledgement, and Reference

- <https://owasp.org/www-project-top-ten/>
- <https://resources.infosecinstitute.com/topics/application-security/7-most-common-application-backdoors/>
- Software Engineering, 10th Edition, Ian Someville
- ISO/SAE 21434 <https://www.iso.org/standard/70918.html>



University
of Glasgow

Thank you!
Any Questions?